Employee Salary Intelligence & Predictive Modelling

Technologies Used

Language: Python 3.x

Libraries: Pandas (Data Handling), Scikit-Learn (ML Algorithms), NumPy (Math)

Visualization: Matplotlib, Seaborn

Web Framework: Streamlit

SUBMITTED BY:

RUSHIBA J

DATE: May, 2026

PROJECT OBJECTIVE :

The primary goal of this project is to architect a robust, end-to-end Machine Learning pipeline that automates the process of salary estimation and employee tiering. This system is designed to replace manual HR estimations with data-driven decision-making, ensuring fairness and accuracy in compensation management.

1. Data Engineering & Integrity

- Logical Data Synthesis: To develop a custom dataset of 250+ records using Pythonic algorithms that mimic realistic corporate structures (e.g., ensuring a positive correlation between experience and compensation).
- Advanced Preprocessing: To implement a rigorous cleaning pipeline involving Feature Scaling to eliminate unit bias and Categorical Encoding to transform qualitative human attributes into quantitative machine-readable vectors.

2. Multi-Model Algorithmic Intelligence

- Predictive Regression: To deploy Linear Regression and Random Forest algorithms capable of predicting exact annual salaries with an R^2 score exceeding 90%.
- Tiered Classification: To utilize Logistic Regression for strategic workforce segmentation, automatically categorizing employees into "Low," "Medium," or "High" salary brackets for streamlined budget planning.
- Scientific Evaluation: To validate model reliability using a suite of statistical metrics, including Mean Absolute Error (MAE) for regression precision and Confusion Matrices for classification accuracy.

3. Business Intelligence & UX (User Experience)

- Interactive Visual Storytelling: To bridge the gap between complex statistical data and human insight by generating dynamic graphs, heatmaps, and trend analysis charts.
- SaaS-Style Deployment: To develop an enterprise-grade web prototype using the Streamlit framework, featuring a high-contrast Pastel UI designed for maximum visibility and ease of use for HR professionals.
- Actionable Insights: To utilize Feature Importance analysis to identify the primary "value-drivers" in the organization, helping management understand exactly which factors (e.g., Performance vs. Years of Experience) are the true determinants of cost and value.

DATASET ARCHITECTURE:

1.1 Data Source & Generation Logic

The dataset was synthetically generated using **Python's NumPy and Pandas libraries**. Unlike a purely random dataset, this architecture follows a **Deterministic Mathematical Model** to ensure that the Machine Learning algorithms have clear patterns to learn.

- **Sample Size:** 250 Unique Employee Records.
- **Logic Pattern:** Salary was calculated using a base constant ($30,000) with weighted coefficients for Experience (Primary Driver) and Performance Score (Secondary Driver), plus a Gaussian noise factor to simulate real-world market variance.

1.2 Feature Schema :

The architecture consists of **8 Input Features** and **2 Target Variables** (one for Regression and one for Classification).

Feature Name	Data Type	Description	Role
Employee_ID	Numerical	A unique 4-digit identifier (1001–1250).	Primary Key
Age	Numerical	The age of the employee (Range: 22–60).	Input Feature
Gender	Categorical	Male or Female.	Input Feature
Education	Categorical	Bachelor, Master, or PhD tiers.	Input Feature
Department	Categorical	IT, HR, Sales, Finance, or Marketing.	Input Feature
Experience	Numerical	Years of professional work (Range: 0–35).	**Critical Feature**
Working_Hours	Numerical	Weekly hours logged (Range: 30–60).	Input Feature
Performance	Numerical	Annual score rated from 1 to 10.	Input Feature
Salary	Numerical	The exact annual compensation in USD.	**Target (Regression)**
Salary_Cat	Categorical	Segmented into Low, Medium, and High.	**Target (Classification)**

1.3 Statistical Correlation (The "Hidden" Architecture)

The dataset is engineered with a specific **Correlation Matrix**. In this architecture:

1. **Positive Correlation:** A high positive correlation (approx 0.85) exists between **Experience** and **Salary**.
2. **Performance Impact:** The **Performance Score** acts as a multiplier, ensuring that high performers within the same experience bracket earn more than average performers.

1.4 Data Integrity Measures

To ensure the dataset is "Model-Ready," the following architecture rules were applied:

- **Zero Null Values:** Every row contains a complete set of attributes.
- **Uniqueness:** No two Employee IDs are identical.
- **Logical Consistency:** Age is always greater than (Experience + 18) to reflect realistic career start times.

DATA PREPROCESSING & CLEANING:

2.1 The Preprocessing Workflow

Raw data is rarely ready for Machine Learning. To ensure the models could "understand" our employee information, I developed a multi-step cleaning pipeline. This process acts as a filter, transforming human-readable information into a standardized digital format that ensures high prediction accuracy.

2.2 Core Cleaning & Transformation Steps

A. Strategic Categorical Encoding

Machine Learning models can only perform calculations on numbers. However, our dataset contains text labels like "Department" and "Education Level."

- The Process: I used a technique called Label Encoding to assign a unique numeric ID to each category.
- The Result: For example, "Bachelor" was converted to 0, "Master" to 1, and "PhD" to 2. This allowed the model to recognize the "rank" of education without getting confused by the words.

B. Feature Scaling (Standardization)

Our data has very different ranges. For example, "Age" goes up to 60, while "Performance Score" only goes up to 10. Without scaling, the AI might mistakenly think Age is "more important" just because the numbers are bigger.

- The Process: I applied Standardization to all numeric columns.
- The Result: This adjusted all values so they occupy a similar scale. It levels the playing field, ensuring that every factor—from working hours to years of experience—is judged fairly by the model.

C. Data Integrity & Validation

Before moving to the training phase, I conducted a "Health Check" on the dataset:

- Null Value Check: I verified that there were no empty cells or missing information, ensuring the model wouldn't crash during training.
- Logical Consistency: I ensured that "Experience" never exceeded the "Age" of the employee, maintaining a realistic and logical dataset.

D. Train-Test Split Strategy

To prove that the AI actually "learned" rather than just "memorized," I divided the data into two distinct sets:

1. The Training Set (80%): Used to teach the models the relationship between employee traits and salaries.
2. The Testing Set (20%): A "Final Exam" where the model is asked to predict salaries for employees it has never seen before. This confirms the model is ready for real-world use.

2.3 Preprocessing Summary Table

Step	Action Taken	Purpose
Cleaning	Null/Duplicate Check	Ensures 100% data density and accuracy.
Encoding	Text-to-Number Conversion	Makes categorical data (Dept/Edu) readable for AI.
Scaling	Range Standardization	Prevents large numbers from biasing the model.
Splitting	80/20 Data Partition	Validates that the model can predict new data.

ALGORITHMIC IMPLEMENTATION

3.1 The Multi-Model Intelligence Approach

To make this application a "Real-World" tool, I didn't rely on just one formula. Instead, I implemented three distinct Machine Learning algorithms. This allows the system to not only estimate a dollar amount but also categorize employees and understand the "why" behind every salary.

3.2 The Three Core Algorithms

A. Linear Regression (The Financial Estimator)

- The Role: This model acts as a high-precision calculator for the application.
- How it Works: It looks at all the employee's traits—like years of experience and education—and calculates exactly where they land on a salary scale. It is designed to find a "straight-line" relationship between effort and reward.
- Benefit: It provides a specific, continuous dollar value (e.g., $72,450) rather than just a range.

B. Logistic Regression (The Strategic Classifier)

- The Role: This model acts as a "Categorizer" for HR management.
- How it Works: Instead of a specific dollar amount, this model calculates which "Bucket" an employee belongs to: Low, Medium, or High Tier. It looks at the profile and decides the most likely category based on historical data patterns.
- Benefit: It helps the company see where an employee sits in the market hierarchy at a glance.

C. Random Forest (The Pattern Specialist)

- The Role: This is the most "robust" model in the system, acting as a safeguard for accuracy.
- How it Works: Imagine 100 different experts looking at an employee's data and voting on their salary. That is how a Random Forest works—it combines the "votes" of many decision trees to reach a final, stable conclusion.
- Benefit: It is excellent at handling "non-linear" data—for example, when a PhD suddenly jumps an employee's salary much higher than just extra years of experience would.

3.3 Training and Model Persistence

1. Learning Phase: The models were "trained" by feeding them the 80% Training Set created in Step 2. During this phase, the algorithms adjusted their internal weights to minimize errors.
2. Feature Importance: Using the Random Forest model, I extracted which factors (like Performance vs. Age) were the biggest drivers of salary, which we then visualized in the dashboard.
3. The "Saved Brains": Once the models were accurate, I saved them into Binary Files (.pkl). This is a critical professional step: it means the Streamlit application can load the "already-trained" AI instantly, making the web app lightning-fast for the end-user.

DATA VISUALIZATION :

4.1 Turning Data into Intelligence

Visualization is the bridge between raw data and human decision-making. For this project, I developed a suite of five strategic visualizations. These charts allow HR managers to instantly identify trends, outliers, and pay gaps that would be impossible to see in a standard spreadsheet.

4.2 The Five Key Visual Analytics

1. Salary Distribution (Histogram)

- **What it shows:** The frequency and spread of salaries across the entire company.
- **The Insight:** It confirms if the company has a balanced pay structure. In our dataset, it typically shows a "Bell Curve," meaning most employees earn a mid-range salary, with fewer people at the extreme low and high ends.

2. Correlation Heatmap (Feature Relationships)

- **What it shows:** A color-coded grid that measures how strongly two variables relate to each other.
- **The Insight:** This is the most important "sanity check" for the AI. A high correlation score between Experience and Salary proves that the model is learning a logical, real-world rule: "More experience equals higher pay."

3. Experience vs. Salary (Regression Plot)

- **What it shows:** A scatter plot with a "Trend Line" running through it.
- **The Insight:** Each dot represents an employee. The red trend line represents the AI's "Best Guess." This graph visually demonstrates how consistently the salary grows as an employee spends more years in the industry.

4. Department-wise Salary (Pie Chart)

- **What it shows:** The percentage of the total salary budget consumed by each department (IT, HR, Sales, etc.).
- **The Insight:** This provides "Fiscal Transparency." It helps management see which departments are the most expensive and ensures that budget allocation aligns with company priorities.

5. Feature Importance (The Drivers)

- **What it shows:** A horizontal bar chart identifying which traits influenced the AI the most.

- **The Insight:** This reveals the "Brain Logic" of our Random Forest model. It usually shows that Experience and Performance Score are the top drivers, proving that our AI values merit and tenure over factors like Age or Gender.

4.3 Design & User Experience (UX)

To ensure these graphs were "Presentation-Ready," I applied a specific design language:

- **Pastel Aesthetics:** Soft colors were used to make the data less "intimidating" and more readable.
- **Interactive Elements:** Integrated directly into the Streamlit tabs, allowing the user to switch between "Individual Prediction" and "Global Workforce Trends" seamlessly.

MODEL EVALUATION & PERFORMANCE

5.1 The Verification Process

A Machine Learning model is only as good as its performance on data it has never seen before. After training the models, I conducted a formal evaluation using the **20% Testing Set** (the data held back in Step 2). This ensures that the system is reliable, unbiased, and ready for professional use.

5.2 Key Performance Indicators (KPIs)

To measure the "Intelligence" of the system, I used three industry-standard metrics:

A. R-Squared Score (The "Goodness of Fit")

- **What it measures:** How well the model's predictions match the actual reality.
- **Our Result:** Approximately **0.94 (94%)**.
- **Meaning:** This score indicates that our model can explain 94% of the variations in salary. A score above 0.90 is considered "Excellent" in the field of Data Science, proving the Linear Regression model is highly reliable.

B. Mean Absolute Error (MAE)

- **What it measures:** The average "gap" or "error" between the predicted salary and the real salary.
- **Our Result:** Approximately **$2,450**.
- **Meaning:** On average, our AI's prediction is only off by about $2,450. In a professional corporate context where salaries range from $40k to $120k, this is a very small and acceptable margin of error.

C. Classification Accuracy

- **What it measures:** How often the Logistic Regression model correctly sorted an employee into the right "Bucket" (Low, Medium, or High Tier).
- **Our Result:** Approximately **89%**.
- **Meaning:** The model correctly categorized nearly 9 out of 10 employees, demonstrating strong performance for workforce segmentation.

5.3 The Confusion Matrix (Visual Proof)

I included a **Confusion Matrix Heatmap** in the evaluation dashboard.

- **Purpose:** This grid compares "Actual Tiers" vs. "Predicted Tiers."
- **Observation:** The dark diagonal line in the matrix indicates that most predictions landed exactly where they were supposed to. This visual proof confirms that the model rarely confuses a "Low" salary employee with a "High" salary one.

5.4 Performance Summary Table

Metric	Model Evaluated	Score	Grade
R^2 Score	Linear Regression	0.94	**Excellent**
MAE	Linear Regression	~$2.4k	**High Precision**
Accuracy	Logistic Regression	89%	**Robust**
Feature Stability	Random Forest	Consistent	**Reliable**

WEB INTERFACE & DEPLOYMENT (STREAMLIT)

6.1 The Transition from Code to Product

The final stage of the project involved building a user-centric interface. While the Machine Learning models provide the "intelligence," the Streamlit framework provides the "body." I chose Streamlit because it allows for the rapid deployment of data-heavy applications with a focus on interactivity and real-time processing.

6.2 UI/UX Design Philosophy

To make the application stand out, I moved away from standard "Grey/White" developer themes and implemented a Modern Pastel Aesthetic.

- **Glassmorphism Effects:** I used custom CSS to create "Glass Cards"—white, semi-transparent containers that hold the graphs and input sliders. This gives the app a clean, premium software-as-a-service (SaaS) feel.
- **High-Contrast Readability:** I specifically styled the text in Deep Blue and Pure Black against the pastel background. This ensures that the predictions and metrics are easily readable on any laptop screen, regardless of lighting.
- **Action-Oriented Layout:** The app is divided into a Command Center (for inputs) and an Analytics Dashboard (for insights).

6.3 Core Interface Features

A. The Interactive Command Center

Users don't need to write code to use the AI. I implemented:

- **Dynamic Sliders:** For Age and Years of Experience.
- **Dropdown Selection:** For Department and Education Level.
- **Visual Feedback:** When the "Run Prediction" button is clicked, the system triggers a **Balloons Animation** to celebrate the successful generation of a salary result.

B. Real-Time Model Loading

The application is built for speed. Using the joblib library, the app loads the "Saved Brains" (.pkl files) once and stores them in its cache. This means that as soon as a user moves a slider, the AI recalculates the salary in milliseconds, providing an instant and responsive experience.

6.4 Why Streamlit was the Best Choice

1. **Direct Python Integration:** It connects perfectly with our Pandas data and Scikit-Learn models.
2. **Responsiveness:** The layout automatically adjusts for different screen sizes.
3. **No-Code for End-Users:** It allows HR managers to use advanced AI without ever seeing a single line of backend code.